forkrun — NUMA-Aware Contention-Free Streaming Parallelization for HPC Data Prep

forkrun is a self-tuning, drop-in replacement for GNU Parallel that accelerates shell-based data preparation by 50×–400× on modern CPUs and scales linearly (or better) on NUMA systems like Frontier.

forkrun achieves:

- **200,000+ batch dispatches/sec** (vs ~500 for GNU Parallel)
- **~95–99% CPU utilization** across all cores (vs ~6% for GNU Parallel)
- **Near-zero cross-socket memory traffic** (NUMA-aware “born-local” design)
- **Automatic recovery and retry** when a worker unexpectedly dies processing a batch

forkrun is built for high-frequency, low-latency workloads on NUMA hardware - a regime where existing tools leave most cores idle.

The Problem

Data preparation on multi-socket HPC systems like Frontier means running millions of fast shell operations — format conversions, field extractions, validation checks, and file transforms — across inputs ranging from a few records to billions of lines. GNU Parallel and `xargs -P` were designed for long-running jobs, not microsecond-scale operations on NUMA hardware. At scale, their per-item fork overhead, cross-socket data migration, and lock contention become the bottleneck — not the work itself.

forkrun, in its fastest mode, can distribute **200 000+ batches/sec** on a single node — while **GNU Parallel struggles to break 500**. On Frontier, this potentially reduces the cost of data prep (measured in total node time) from over 50% down to under 10%.

What forkrun Is

forkrun is an **intra-node** drop-in shell parallelizer that replaces `xargs -P` and GNU Parallel for streaming workloads on a single machine. It is easy to use — source the script, and it can immediately parallelize native bash functions or external commands:

```
 . frun.bash                                  # sourcing frun.bash sets up *everything*
frun my_bash_func < inputs.txt               # parallelize custom bash functions!
cat file_list | frun -k sed 's/old/new/'     # pipe-based input, ordered output
frun -k -s sort < records.tsv                # stdin-passthrough, ordered output
frun -s -I 'gzip -c >{ID}.gz' < raw_logs     # stdin-passthrough, unique output names
```

Under the hood, forkrun is a **contention-free, NUMA-aware, dynamically self-tuning parallelization engine** implemented as a set of C loadable bash builtins. It coordinates workers through shared memory and atomic operations — no locks on the fast path, no cross-socket data migration, no per-item fork overhead.

How It Works

The data pipeline has four stages, each designed to preserve locality:

1. **Ingest**: Data is `splice()`'d from stdin into a shared memfd. This is **PFS-friendly**, multiplexing data entirely in kernel space without generating filesystem metadata storms (no `stat()`/`open()` cascades). On multi-socket systems, `set_mempolicy(MPOL_BIND)` places each chunk's pages on a target NUMA node *before any worker touches them*. This placement is driven by real-time backpressure from the per-node indexers, making NUMA distribution completely self-load-balancing. Data is always **born-local**.
2. **Index**: Per-node indexers (pinned to their socket) find record boundaries using AVX2/NEON SIMD scanning at memory bandwidth, dynamically batch based on runtime conditions, then publish offset markers into a per-node lock-free ring buffer.
3. **Claim**: Workers claim batches via a single `atomic_fetch_add` — no CAS retry loops, no locks, no contention. Workers who overshoot deposit remainders into an escrow pipe for idle workers to steal.
4. **Reclaim**: A background fallow thread punches holes behind completed work via `fallocate(PUNCH_HOLE)`, bounding memory usage without breaking the offset coordinate system.

Adaptive tuning is fully automatic. A three-phase controller (warmup → geometric ramp → PID steady-state) discovers the optimal batch size in O(log L) steps and continuously adjusts based on input rate, consumption rate, and worker starvation — with no user configuration required. forkrun runs efficiently whether it has 20 inputs from `ping` running on your laptop, or a billion lines from a file on a ramdisk running on a Frontier node.

Benchmarks (14-core/28-thread i9-7940x, 100 M lines)

Workload	forkrun	GNU Parallel	Speedup	Notes
Default (array + fully-quoted args, no-op)	**25.0 M lines/s**	58 k lines/s	**~430×**	forkrun default mode
Ordered output (`-k` , no-op)	**24.5 M lines/s**	57 k lines/s	**~430×**	ordering is free in forkrun
`echo` (line args)	**22.6 M lines/s**	~55 k lines/s	**~410×**	typical shell command
`printf '%s\n'` (I/O heavy)	**12.8 M lines/s**	~58 k lines/s	**~220×**	formatting + output
`-s` stdin passthrough (no-op)	**1.04 B lines/s**	6.05 M lines/s (`--pipe`)	**~172×**	streaming / splice
`-b 512k` byte batches (no-op)	**2.51 B lines/s**	6.02 M lines/s (`--pipe`)	**~417×**	kernel-limited

NOTE: All benchmarks run on UMA hardware booted with `numa=fake=4` . On NUMA hardware, forkrun is expected to scale linearly (or better).

Test Coverage & Validation

- forkrun has been rigorously validated with **3,840 successful test runs**: (244 unit tests + 396 benchmark runs) × (UMA + NUMA) × (baseline + TSan + ASan/UBSan)

Batch distribution rate

- forkrun default mode: **~10 000 – 12 000 batches/sec**
- forkrun `-s` mode: **> 200 000 batches/sec (UMA) / > 100 000 batches/sec (NUMA)**
- GNU Parallel (current tool): **~470 batches/sec**

Average CPU utilization across ~400 benchmarks

- forkrun: 95% (27.1 / 28 cores) (no centralized dispatcher - all 27.1 cores doing work)
- GNU Parallel: 6% (2.68 / 28 cores) (1 full core used strictly for dispatching work - 1.68 cores doing actual work)

Comparison of forkrun Modes

- **`-s` mode** is the headline: data flows memfd → kernel pipe → command stdin via `splice()`, entirely in kernel space. Bash never touches the data bytes — only the claim/dispatch coordination runs in userspace.
- **`-b` mode**: allows for distributing batches of constant byte size without needing to scan for delimiters. Performance approaching kernel limits on memory movement.
- **`-k` mode (Ordered output)**: has virtually zero cost. Benchmarks indicate that ordering adds under 2% to the runtime, whereas strict ordering brutally penalizes traditional tools.
- **CPU utilization**: avg 27.1 / 28 cores (95.2%) sustained across all modes for ~400 tests. "Default" mode tests saturate on avg 27.6 / 28 cores (98.6%).
- **Cross-socket traffic (NUMA, 4 nodes)**: 0.0–0.2% of chunks — born-local placement works and cross-node traffic is virtually eliminated.
- **File vs pipe input**: zero measurable difference — the ingest pipeline handles both identically.

Key Design Properties

- **Contention-free**: The fast path is intentionally boring and excessively fast (a single atomic increment with no locks or CAS retry loops). All algorithmic complexity is shifted to the slow path to ensure graceful degradation, meaning contention is structurally eliminated rather than reactively avoided.
- **Born-local NUMA**: Data is placed on the correct socket at ingest time via `set_mempolicy` using real-time backpressure (self load-balancing). Scanners and workers are pinned. Cross-socket traffic is a measured 0.0–0.2%. Stealing is permitted only when local work is exhausted.
- **Zero-copy data path**: `splice()`, `copy_file_range()`, and `sendfile()` move data without userspace copies. Scanner publishes byte-offsets and line counts. Workers read directly from the backing memfd.
- **Self-tuning**: Automatic worker scaling, adaptive batch sizing, and early partial flush for low-latency trickle inputs. No manual `-n` or `-j` tuning required.
- **Fault-tolerant & Self-healing**: Built-in automatic recovery for unexpectedly killed workers (e.g., OOM kills, segfaults). `forkrun` automatically traps the failure, isolates and discards corrupted partial output, safely respawns the worker, and re-dispatches the poisoned batch without deadlocking the pipeline.

- **Single-file deployment**: Ships as one bash file with an embedded loadable `.so`. Zero external dependencies beyond a handful of standard Linux utilities (e.g., sed, base64, gzip, rm, cat) — no heavy runtimes like Perl (unlike GNU Parallel) or Python, making it perfect for lightweight containerized deployments. Requires only a Linux kernel ≥ 3.17 and Bash ≥ 4.0 (Bash ≥ 5.1 recommended).
- **Secure & Verifiable Deployment**: Ships as one bash file with an embedded loadable .so. The binary is compiled and injected automatically via GitHub Actions, providing an auditable cryptographic trail from the C source code to `frun.bash` —meeting strict HPC facility security requirements.

Why It Matters for Frontier: Data Prep

forkrun targets a known inefficiency in HPC workflows: underutilized CPUs during data preparation.

Frontier's compute nodes rely on customized 64-core AMD EPYC "Trento" CPUs configured with 4 NUMA domains (NPS4). Data prep workflows that run millions of fast shell transforms hit exactly the failure mode that forkrun was designed for: **high-frequency, low-latency operations on deep NUMA topologies**.

GNU Parallel's per-item Perl initialization overhead and NUMA-oblivious scheduling leave most cores idle on this workload shape. forkrun keeps them saturated with node-local data. On systems like Frontier, where data prep can dominate runtime, this represents a **significant opportunity for reclaiming compute capacity**.

Current Limitations & Roadmap for Resilience

While `forkrun` now features robust intra-node fault tolerance (automatically recovering from individual worker crashes without data loss), transitioning it into a hardened, facility-wide utility requires advancing its cluster-level and system-state capabilities. Priorities for the development roadmap include:

- **Resume-after-interruption** state saving to gracefully handle preempted Slurm jobs without losing progress.
- **Deeper integration** with facility workload managers to dynamically expand or contract resource usage.

Executing this roadmap, hardening the codebase for Exascale production environments, and providing dedicated facility support is the primary focus for proposed collaboration and funding with ORNL.

Next steps / Contact / Source

forkrun is open source (MIT License). Drop `frun.bash` on a Frontier login node and run `. frun.bash && frun -s : < 1B_line_file` side-by-side with your current Parallel pipeline. I'm happy to assist remotely or on-site. I live in Dandridge, TN (~1 hour away from ORNL) and am available for an on-site demo with minimal notice.

Let's work together to get Frontier spending **more time doing science** and less time "waiting for data".

Anthony Barone

BSc Geophysics (UC Berkeley) • MSc Geophysics (UT Austin — advised by Mrinal Sen)
Dandridge, TN (1 hour from ORNL) • anthonywbarone@gmail.com • (858) 735-2342
https://github.com/jkool702/forkrun • Background: Computational Geophysics & Inverse Theory